

CS302 OS Assignment-1 Report

12312110 李轩然

DDL: Apr.6 16:00

Time spent: about 4h

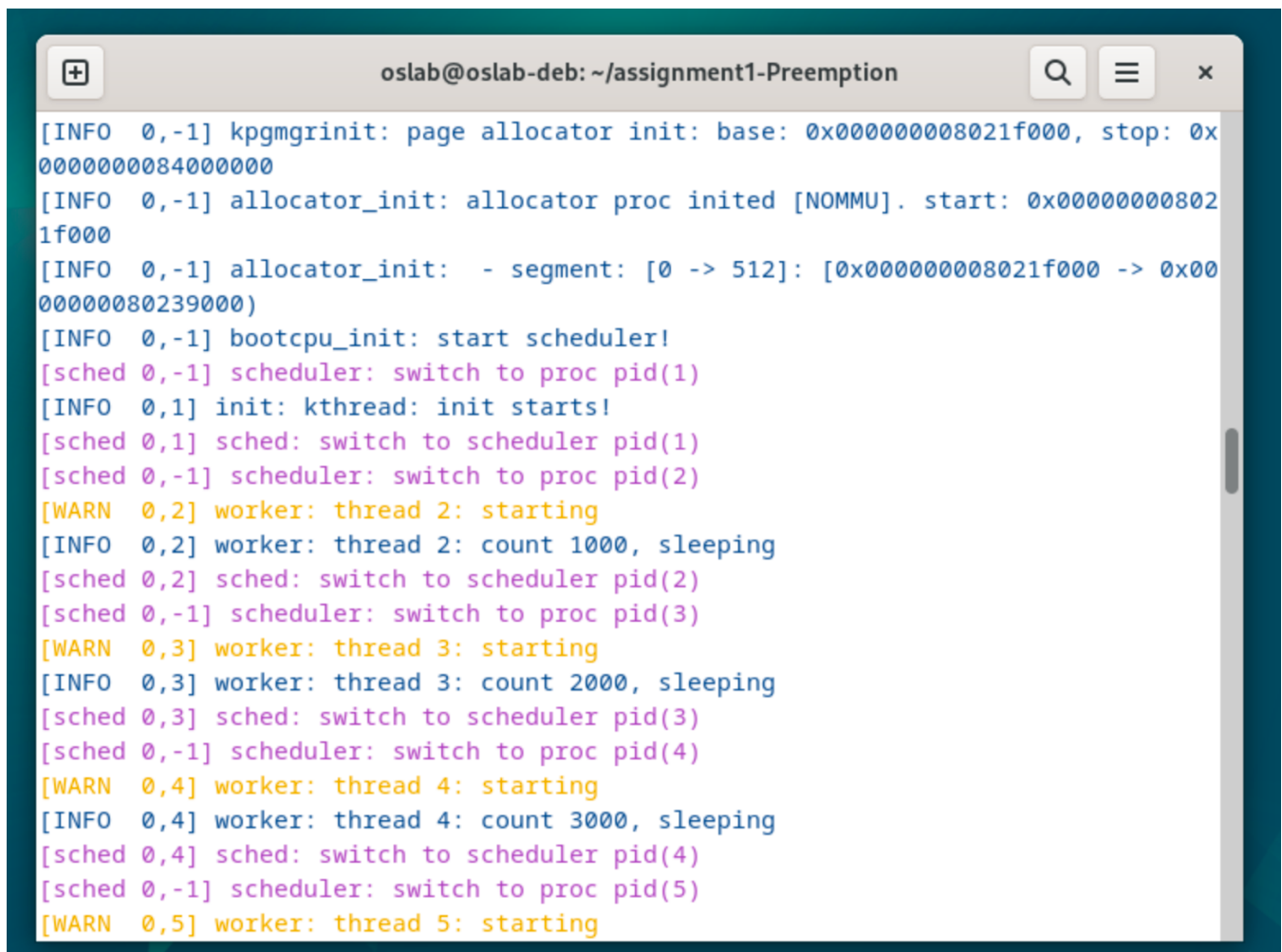
Checkpoint 1

We change the `SupervisorTimer` in `trap.c`:

```
case SupervisorTimer:
    tracef("s-timer interrupt, cycle: %d", r_time());
    set_next_timer();

    struct proc *p = curr_proc();
    if (p != NULL) {
        int saved_inkernel_trap = mycpu()->inkernel_trap;
        mycpu()->inkernel_trap = 0;
        yield();
        mycpu()->inkernel_trap = saved_inkernel_trap;
    }
    break;
```

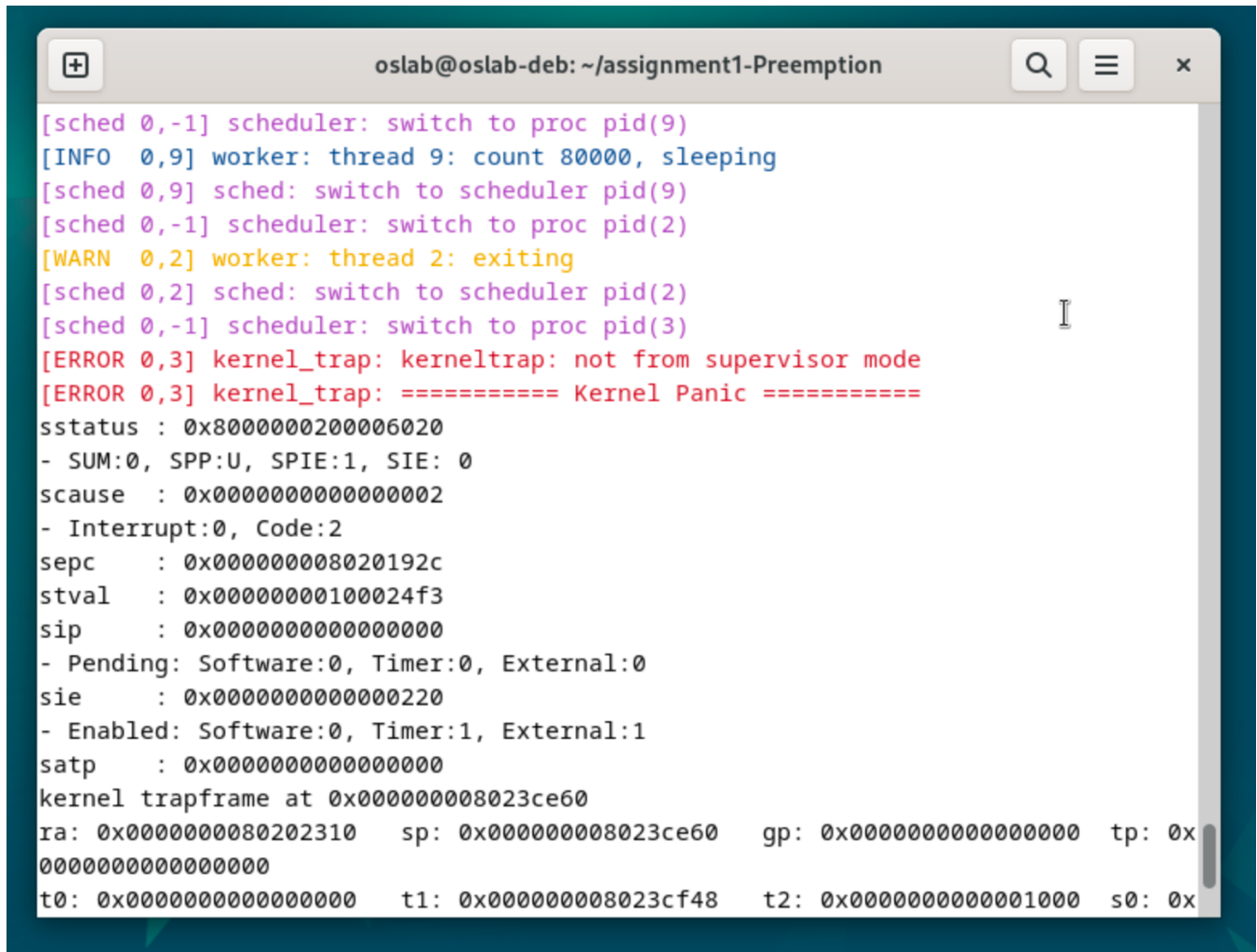
The result is as follows:

A screenshot of a terminal window titled "oslab@oslab-deb: ~/assignment1-Preemption". The terminal displays a series of log messages from an operating system boot process. The messages include information about page allocator initialization, scheduler initialization, and the starting of worker threads. The logs show a sequence of events where the scheduler switches between different processes (pid(1) to pid(5)) and worker threads (thread 2 to thread 5) are started and put to sleep. The messages are color-coded: blue for general info, purple for scheduler actions, and yellow for worker thread actions.

```
[INFO  0,-1] kpgmgrinit: page allocator init: base: 0x000000008021f000, stop: 0x0000000084000000
[INFO  0,-1] allocator_init: allocator proc inited [NOMMU]. start: 0x000000008021f000
[INFO  0,-1] allocator_init: - segment: [0 -> 512]: [0x000000008021f000 -> 0x0000000080239000)
[INFO  0,-1] bootcpu_init: start scheduler!
[sched 0,-1] scheduler: switch to proc pid(1)
[INFO  0,1] init: kthread: init starts!
[sched 0,1] sched: switch to scheduler pid(1)
[sched 0,-1] scheduler: switch to proc pid(2)
[WARN  0,2] worker: thread 2: starting
[INFO  0,2] worker: thread 2: count 1000, sleeping
[sched 0,2] sched: switch to scheduler pid(2)
[sched 0,-1] scheduler: switch to proc pid(3)
[WARN  0,3] worker: thread 3: starting
[INFO  0,3] worker: thread 3: count 2000, sleeping
[sched 0,3] sched: switch to scheduler pid(3)
[sched 0,-1] scheduler: switch to proc pid(4)
[WARN  0,4] worker: thread 4: starting
[INFO  0,4] worker: thread 4: count 3000, sleeping
[sched 0,4] sched: switch to scheduler pid(4)
[sched 0,-1] scheduler: switch to proc pid(5)
[WARN  0,5] worker: thread 5: starting
```

Figure 1

Figure 1 shows that 8 worker threads run in turns.

A terminal window titled 'oslab@oslab-deb: ~/assignment1-Preemption' displays kernel logs. The logs show a sequence of events: a scheduler switch to proc pid(9), a worker thread 9 sleeping, a scheduler switch to scheduler pid(9), a scheduler switch to proc pid(2), a warning that worker thread 2 is exiting, a scheduler switch to scheduler pid(2), a scheduler switch to proc pid(3), and then a kernel trap error: 'kernel_trap: kerneltrap: not from supervisor mode' followed by 'kernel_trap: ===== Kernel Panic ====='. Below the error, detailed system status is printed, including sstatus, scause, Interrupt, sepc, stval, sip, Pending, sie, Enabled, satp, and a kernel trapframe with registers ra, sp, gp, tp, t0, t1, t2, and s0.

```
[sched 0,-1] scheduler: switch to proc pid(9)
[INFO 0,9] worker: thread 9: count 80000, sleeping
[sched 0,9] sched: switch to scheduler pid(9)
[sched 0,-1] scheduler: switch to proc pid(2)
[WARN 0,2] worker: thread 2: exiting
[sched 0,2] sched: switch to scheduler pid(2)
[sched 0,-1] scheduler: switch to proc pid(3)
[ERROR 0,3] kernel_trap: kerneltrap: not from supervisor mode
[ERROR 0,3] kernel_trap: ===== Kernel Panic =====
sstatus : 0x80000000200006020
- SUM:0, SPP:U, SPIE:1, SIE: 0
scause : 0x0000000000000002
- Interrupt:0, Code:2
sepc : 0x000000008020192c
stval : 0x00000000100024f3
sip : 0x0000000000000000
- Pending: Software:0, Timer:0, External:0
sie : 0x0000000000000220
- Enabled: Software:0, Timer:1, External:1
satp : 0x0000000000000000
kernel trapframe at 0x000000008023ce60
ra: 0x0000000080202310 sp: 0x000000008023ce60 gp: 0x0000000000000000 tp: 0x
0000000000000000
t0: 0x0000000000000000 t1: 0x000000008023cf48 t2: 0x0000000000001000 s0: 0x
```

Figure 2

Figure 2 shows that we encountered the error *kerneltrap: not from supervisor mode* followed by a kernel panic.

Checkpoint 2

We change the `SupervisorTimer` in `trap.c` again:

```
case SupervisorTimer:
    tracef("s-timer interrupt, cycle: %d", r_time());
    set_next_timer();

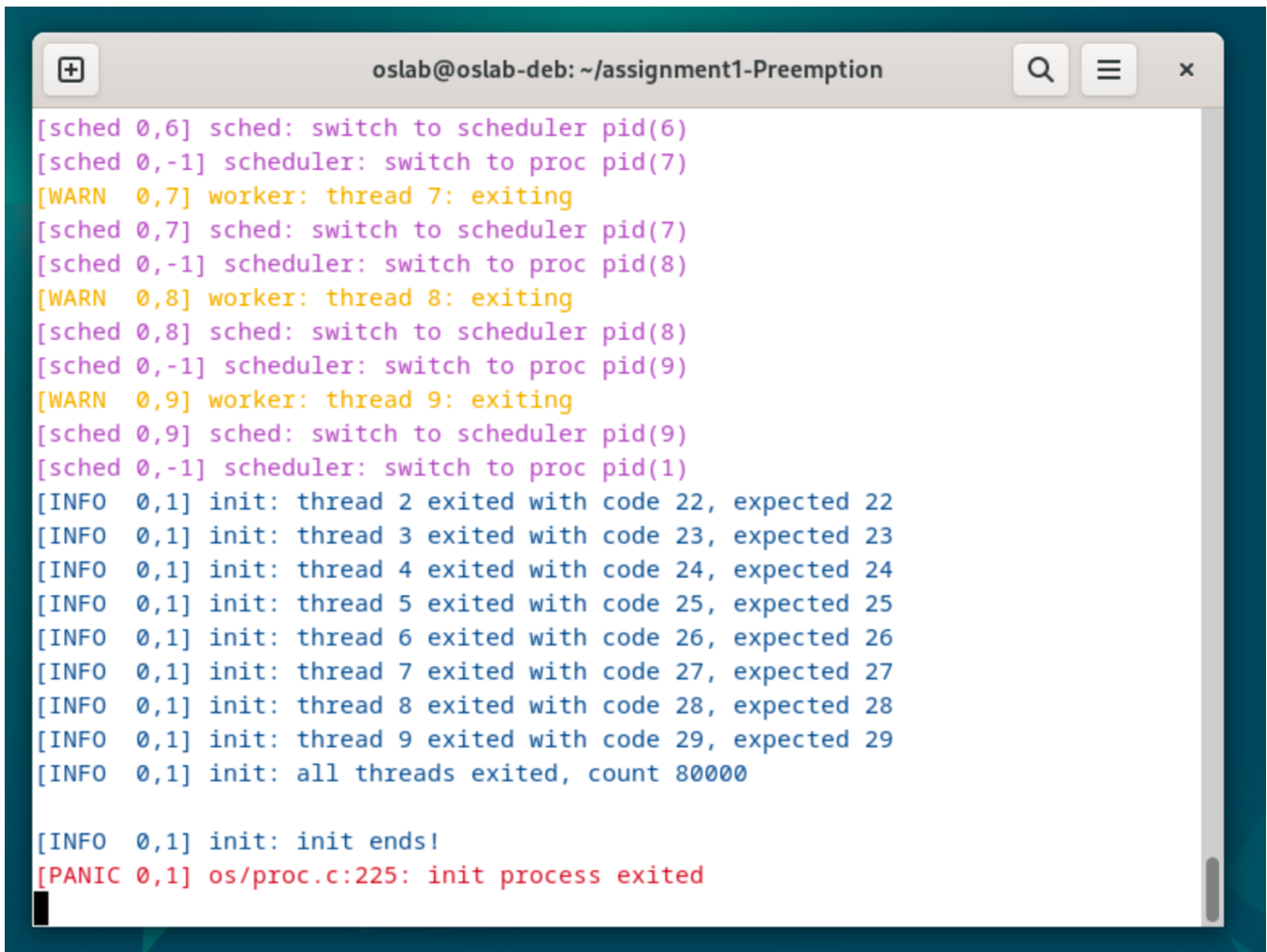
    struct proc *p = curr_proc();
    if (p != NULL) {
        uint64 saved_sepc = r_sepc();
        uint64 saved_sstatus = r_sstatus();
        int saved_inkernel_trap = mycpu()->inkernel_trap;

        mycpu()->inkernel_trap = 0;
        yield();
        mycpu()->inkernel_trap = saved_inkernel_trap;

        w_sepc(saved_sepc);
```

```
w_sstatus(saved_sstatus);  
}  
break;
```

The result is as follows:



```
oslab@oslab-deb: ~/assignment1-Preemption  
[sched 0,6] sched: switch to scheduler pid(6)  
[sched 0,-1] scheduler: switch to proc pid(7)  
[WARN 0,7] worker: thread 7: exiting  
[sched 0,7] sched: switch to scheduler pid(7)  
[sched 0,-1] scheduler: switch to proc pid(8)  
[WARN 0,8] worker: thread 8: exiting  
[sched 0,8] sched: switch to scheduler pid(8)  
[sched 0,-1] scheduler: switch to proc pid(9)  
[WARN 0,9] worker: thread 9: exiting  
[sched 0,9] sched: switch to scheduler pid(9)  
[sched 0,-1] scheduler: switch to proc pid(1)  
[INFO 0,1] init: thread 2 exited with code 22, expected 22  
[INFO 0,1] init: thread 3 exited with code 23, expected 23  
[INFO 0,1] init: thread 4 exited with code 24, expected 24  
[INFO 0,1] init: thread 5 exited with code 25, expected 25  
[INFO 0,1] init: thread 6 exited with code 26, expected 26  
[INFO 0,1] init: thread 7 exited with code 27, expected 27  
[INFO 0,1] init: thread 8 exited with code 28, expected 28  
[INFO 0,1] init: thread 9 exited with code 29, expected 29  
[INFO 0,1] init: all threads exited, count 80000  
  
[INFO 0,1] init: init ends!  
[PANIC 0,1] os/proc.c:225: init process exited
```

Figure 3

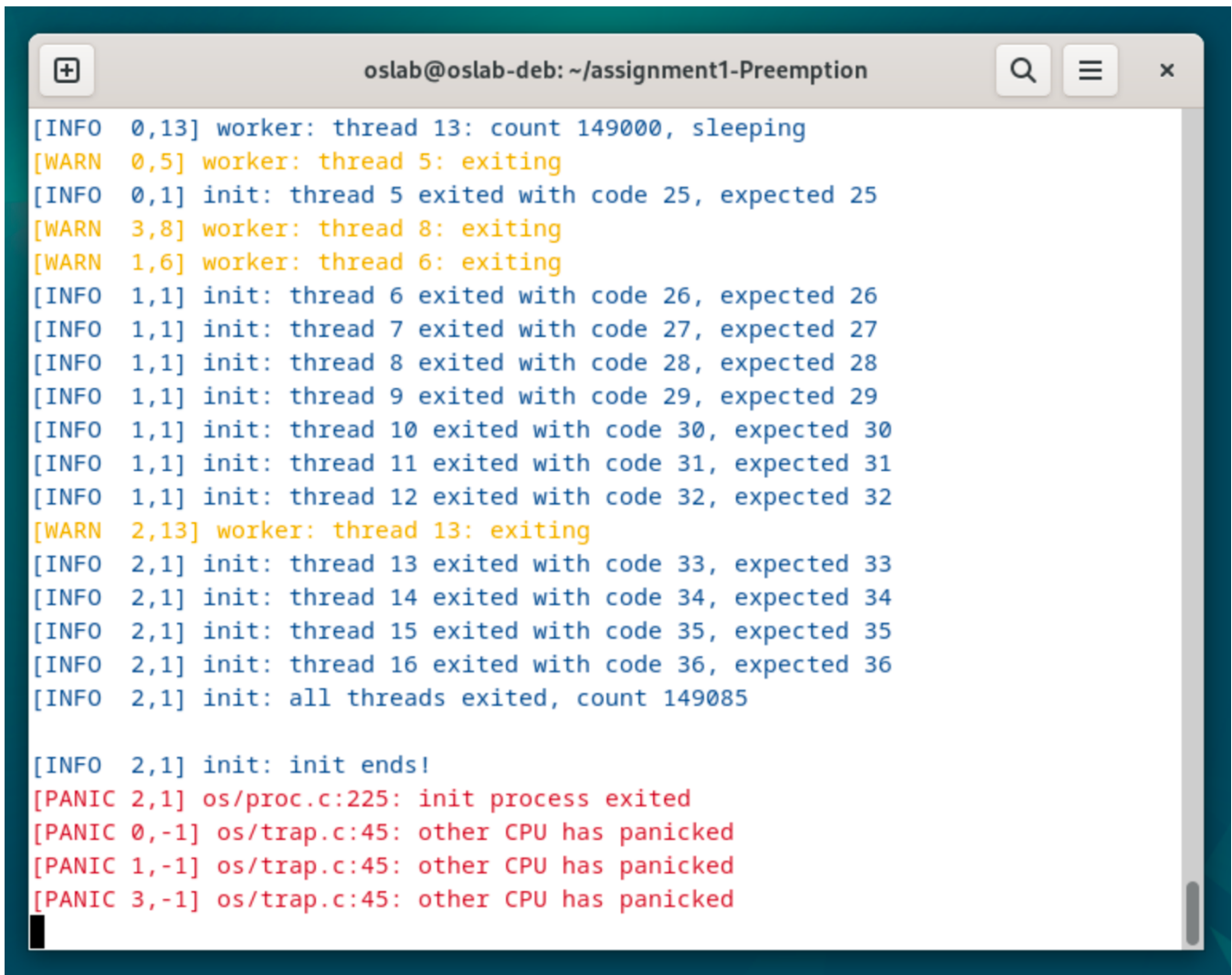
Figure 3 shows that we executed the `init` function on a single CPU.

Checkpoint 3

Changes:

```
os/sched.c  
// logf(PURPLE, "sched", "switch to proc pid(%d)", p->pid);  
// logf(PURPLE, "sched", "switch to scheduler pid(%d)", p->pid);  
  
os/timer.h  
#define TICKS_PER_SEC (400)  
  
os/nommu_init.c  
#define NTHREAD 15  
#define SLEEP_TIME 50
```

The result is as follows:

A terminal window titled "oslab@oslab-deb: ~/assignment1-Preemption" displays a series of kernel log messages. The logs show the progression of thread initialization and exit, with counts and expected values. The messages are color-coded: INFO in blue, WARN in yellow, and PANIC in red. The sequence of events includes worker threads sleeping and exiting, followed by a series of init threads exiting with specific codes. The process concludes with a final count of 149085 and a series of panic messages from other CPUs.

```
[INFO 0,13] worker: thread 13: count 149000, sleeping
[WARN 0,5] worker: thread 5: exiting
[INFO 0,1] init: thread 5 exited with code 25, expected 25
[WARN 3,8] worker: thread 8: exiting
[WARN 1,6] worker: thread 6: exiting
[INFO 1,1] init: thread 6 exited with code 26, expected 26
[INFO 1,1] init: thread 7 exited with code 27, expected 27
[INFO 1,1] init: thread 8 exited with code 28, expected 28
[INFO 1,1] init: thread 9 exited with code 29, expected 29
[INFO 1,1] init: thread 10 exited with code 30, expected 30
[INFO 1,1] init: thread 11 exited with code 31, expected 31
[INFO 1,1] init: thread 12 exited with code 32, expected 32
[WARN 2,13] worker: thread 13: exiting
[INFO 2,1] init: thread 13 exited with code 33, expected 33
[INFO 2,1] init: thread 14 exited with code 34, expected 34
[INFO 2,1] init: thread 15 exited with code 35, expected 35
[INFO 2,1] init: thread 16 exited with code 36, expected 36
[INFO 2,1] init: all threads exited, count 149085

[INFO 2,1] init: init ends!
[PANIC 2,1] os/proc.c:225: init process exited
[PANIC 0,-1] os/trap.c:45: other CPU has panicked
[PANIC 1,-1] os/trap.c:45: other CPU has panicked
[PANIC 3,-1] os/trap.c:45: other CPU has panicked
```

Figure 4

We can consistently pass the make runsmpt test more than 10 times and get the logs that Figure 4 shows.